Three Linux operating modes

A supplied PEM pair takes priority. Only an absent pair selects the configured privileged mechanism.

Invoke craft as the user

Real UID, NSS name and home
No arguments; no sudo

Check user.pem and user.key in the NSS home

~/.config/craft/user.pem + user.key
Complete pair takes priority on every installation

BOTH FILES ABSENT

COMPLETE PAIR

Partial, unsafe or invalid:
fail and retain cache.

Select the configured privileged mechanism

Requires setuid launcher, craft service account and /run/craft

1 Privileged enrollment

mechanism=enrollment
Service account: GC + agent + CES
CA issues temporary user certificate
PKINIT authenticates that certificate
CA enforces recipient restrictions

2 Privileged credential linking

mechanism=key_trust
Service account: writable DC + keytab
Add temporary msDS-KeyCredentialLink
Key Trust PKINIT; remove the entry
AD enforces attribute-write scope

3 Unprivileged Windows PEM

Windows: user enrollment + export
Securely transfer certificate and key
Linux: caller validates and uses pair
PKINIT uses KDC account mapping
Supplied PEM files remain in place

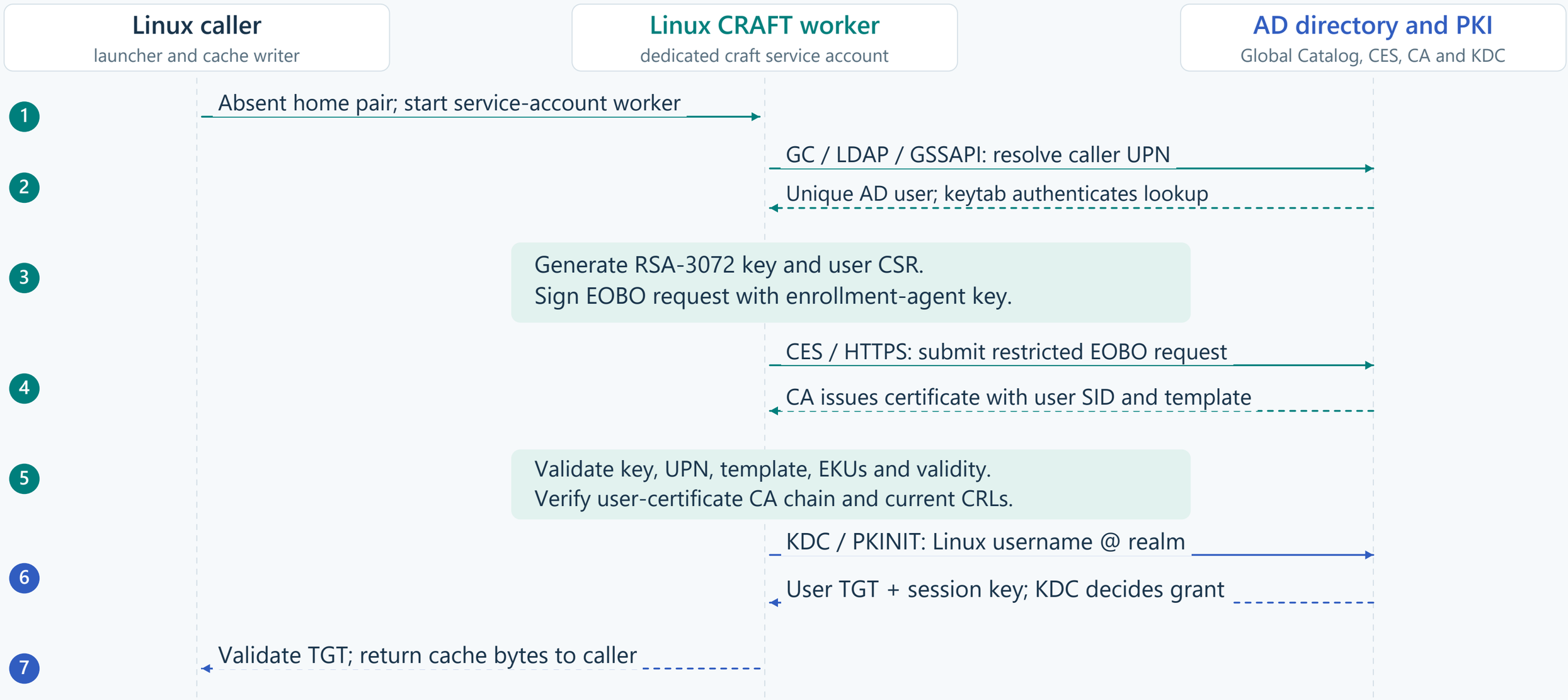
Validate the user TGT and publish the cache as the caller

Fixed principal: Linux username @ realm; .krb5cc_craft mode 0600; select with KRB5CCNAME

SOURCE source/README.md: Mode selection and process boundaries; mode-specific setup recipes

Mode 1 Privileged certificate enrollment

Linux orchestrates CA enrollment first, then PKINIT. Select mechanism=enrollment with both home PEM files absent.

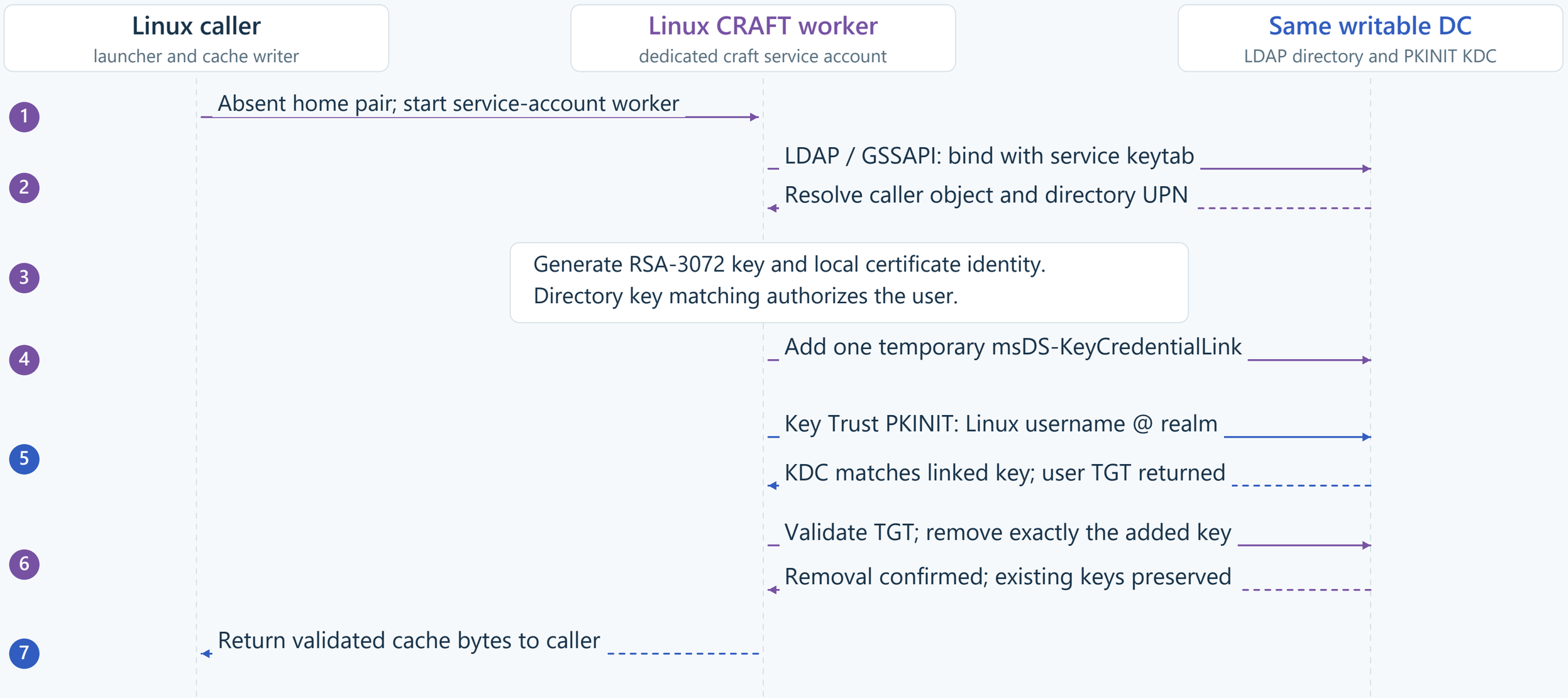


Caller publishes .krb5cc_craft mode 0600. Temporary user key and certificate are released.

SOURCE source/README.md: Mode 1 setup; docs/03-Enrollment-Agent-Restrictions.md

Mode 2 Privileged credential linking

Linux links a temporary AD public key, performs Key Trust PKINIT, and removes the entry before publishing the TGT.



Caller publishes .krb5cc_craft mode 0600. Cleanup also runs after acquisition failure.
Failed cleanup blocks publication and reports a residual key for administrator removal.

SOURCE source/README.md: Mode 2 setup; docs/04-Key-Credential-Link-Delegation.md

Mode 3 Unprivileged Windows-exported certificate

Windows enrolls and exports the certificate. Linux uses the transferred PEM pair entirely as the caller.

Windows domain user
direct user enrollment and export

Linux user and CRAFT
ordinary executables; caller UID

AD CS and KDC
certificate issuance and PKINIT

1. Windows enrollment policy: request a user certificate with an exportable software key →
   ← AD CS issues the user certificate; private key stays with the user
2. Request-CRAFT-Certificate.cmd exports user.pem + matching unencrypted user.key
3. Securely transfer both PEM files to Linux →
4. Install at NSS home: ~/.config/craft/user.pem + user.key
   Run craft as the user; check files and certificate.
   Validate CA chain and CRLs; use public /etc/craft policy.
5. KDC / PKINIT: Linux username @ realm →
   ← KDC maps certificate and returns user TGT
6. Validate returned TGT; publish mode-0600 .krb5cc_craft.
   PEM files remain; replace the pair before expiry.

No Linux setuid bit, service account, directory keytab, LDAP lookup, CES or /run/craft.

SOURCE source/README.md: Mode 3 setup; scripts/Request-CRAFT-Certificate.cmd

Ticket clocks and certificate storage

All three modes request 10 h / 7 d by default. Certificate lifetime and KDC policy constrain the initial grant.

1 / ENROLLMENT
Temporary key and certificate.
CA issuance records remain.

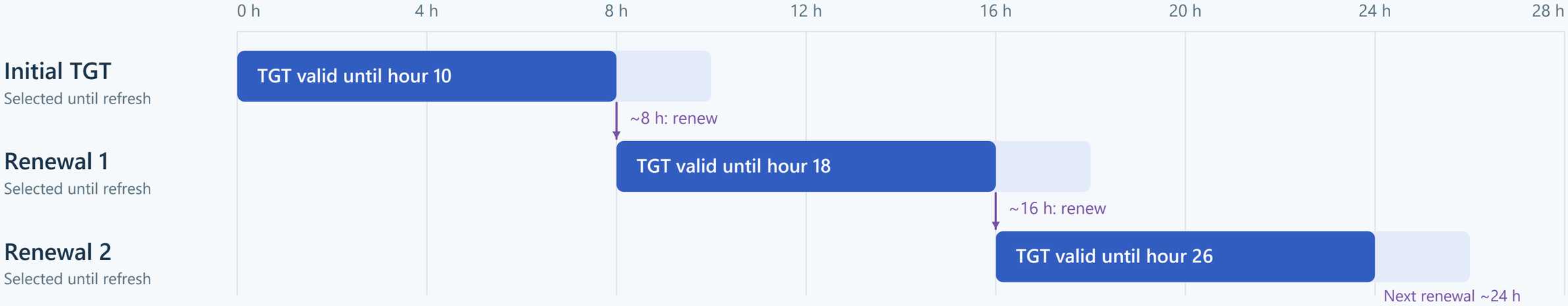
2 / CREDENTIAL LINKING
Temporary key and local identity.
Remove directory entry after PKINIT.

3 / WINDOWS-EXPORTED PEM
Persistent supplied certificate and key.
Replace the pair before expiry.

01 / HOURS

Renew before expiry

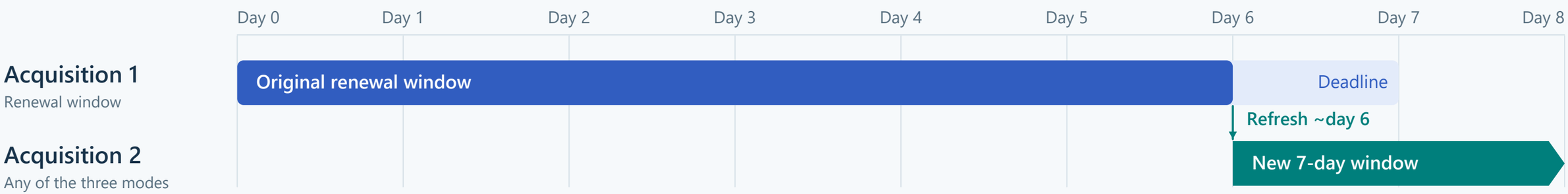
Illustrative full 10 h grants



Renewal advances expiry; renew-till stays fixed.

02 / DAYS

Start a fresh cycle near the renewal deadline



Renewal uses the cached ticket and session key. Certificate expiry or cleanup does not revoke an issued ticket.

SOURCE maintain.cpp: schedule, validate_renewed_tgt; worker.cpp: get_tgt; config.example: lifetime defaults

Keep a job authenticated, then stop with it.

craft-maintain runs as the caller, coordinates one shared cache per UID, and watches a live process.

START ONCE / WATCH A SHELL THAT STAYS ALIVE FOR THE JOB

```
job_pid=$$
cache=$(/usr/local/bin/craft-maintain --watch-pid "$job_pid") || exit "$?"
export KRB5CCNAME="$cache"
```

Returns when usable credentials and background maintenance are ready.

01 / RENEW

Use the current ticket

When a renewable TGT has <=20% of its lifetime left (capped at 2 h), and its expiry can extend.

A renewal error retries; no immediate fresh authentication.

Cached TGT + session key → **KDC** renew request → **Validated TGT** new expiry

Renew-till never moves later.
Retain usable service tickets.

02 / FRESH AUTHENTICATION

Obtain fresh credentials

Missing or expired TGT; renewal deadline is near; or a ticket that cannot extend is nearing expiry.

Rollover margin: 1/7 of the renewal window, capped at 1 day.

Run craft select source → **Selected mode** 1, 2 or 3 → **KDC / PKINIT** new user TGT

Home PEM, enrollment or credential linking.
New renewal window; replace cache.

A due renewal takes priority during rollover, keeping usable credentials alive while fresh authentication is delayed.

Successful refresh

Publish atomically to .krb5cc_craft (caller-owned, mode 0600).

Refresh failure

Keep the existing cache and retry; the job continues.

RETRY

Renewal: ~30 s to 5 min + jitter.
Fresh acquisition: 1 min to 1 h cooldown.

STOP

Watched PID exits, duration ends or signal.
Maintainer stops; the shared cache remains.

APPLICATION

Reload credentials when authenticating again.
Access follows the user's service permissions.

SOURCE maintain.cpp: schedule, update, renewed_cache, detached loop; common.hpp: publish_cache; source/README.md: Long-running jobs